ROMA SUPPLY E-COMMERCE & MOBILE APP

MANUAL TÉCNICO COMPLETO Y GUÍA DE EJECUCIÓN PASO A PASO

DOCUMENTACIÓN TÉCNICA OFICIAL E INSTRUCCIONES DE EJECUCIÓN
Este documento contiene la especificación técnica integral, esquemas de base de datos NoSQL, reglas de seguridad de Firestore, backend serverless, código fuente de la aplicación móvil en Flutter, notificaciones push FCM, pipeline de CI/CD en GitHub Actions y la hoja de ruta de desarrollo de 10 semanas acordados para el proyecto RomaSupply.

Módulo / Área	Tecnología Principal	Estado & Cobertura
Base de Datos NoSQL	Cloud Firestore	Esquema completo con subcolecciones e índices
Backend Serverless	Firebase & GCP Cloud Functions	Transacciones atómicas en TypeScript y Signed URLs
Seguridad & Permisos	Firestore Rules (Zero Trust)	Auditoría de reglas y tests unitarios en Jest/Emulador
App Móvil Multiplataforma	Flutter (iOS & Android)	Clean Architecture Feature-First, GoRouter, Riverpod
Notificaciones Push	Firebase Cloud Messaging (FCM)	Canal Drops Global, transaccionales y deep links
Integración Continua	GitHub Actions	Pipeline CI/CD automatizado para tests, APK, AAB e IPA
Hoja de Ruta de Ejecución	Master Implementation Plan	Plan secuencial de 10 semanas estructurado en 7 fases

1. ARQUITECTURA DE BASE DE DATOS NOSQL (CLOUD FIRESTORE)

La base de datos está diseñada sobre Cloud Firestore optimizando las lecturas en el cliente y garantizando baja latencia en la tienda móvil y web. A continuación se detallan las colecciones raíz y sus esquemas.

1.1 Mapa de Colecciones (Tree View)

```
cloud-firestore-root/
├── products/ {productId}             [Colección Raíz - Catálogo General]
│   └── variants/ {variantId}        [Subcolección - Tallas, Colores y Stock]
├── users/ {uid}                      [Colección Raíz - Sincronizada con Firebase Auth]
│   └── saved_addresses/ {addressId} [Subcolección - Libreta de Direcciones]
├── carts/ {cartId / userId}          [Colección Raíz - Carrito Persistente Web/App]
├── orders/ {orderId}                 [Colección Raíz - Transacciones del Checkout]
├── looks/ {lookId}                   [Colección Raíz - Cross-Selling Get the Look]
└── roma_club/ {emailHash}            [Colección Raíz - Lead Magnet / Suscripciones]
```

1.2 Esquema JSON de Colecciones Clave

Colección `/products` y Subcolección `/variants`:

```
// /products/{productId}
{
  "id": "prod_8829", "title": "Hoodie Oversize Roma", "slug": "hoodie-oversize-roma",
  "category": "streetwear", "price": 89.00, "compare_at_price": 110.00,
  "badge": "NUEVO", "is_active": true, "in_stock": true,
  "images": { "flat_lay": "https://cdn.romasupply.com/hd-flat.jpg", "gallery": [...] },
  "attributes": { "sizes": ["S", "M", "L"], "colors": ["Negro Carbono", "Blanco Puro"] }
}
// /products/{productId}/variants/{variantId}
{
  "id": "var_101", "product_id": "prod_8829", "sku": "RS-HD-001-BLK-M",
  "size": "M", "color": "Negro Carbono", "price": 89.00, "stock": 14
}
```

Colección `/carts` y `/orders` (Snapshot Inmutable):

```
// /carts/{cartId} (cartId == userId)
{
  "cart_id": "usr_993812", "user_id": "usr_993812", "currency": "USD",
  "items": [{ "variant_id": "var_101", "product_id": "prod_8829", "title": "Hoodie Oversize Roma", "size": "
M", "color": "Negro Carbono", "price": 89.00, "quantity": 1 }],
  "subtotal": 89.00
}
// /orders/{orderId}
{
  "order_id": "ord_2026_8841", "user_id": "usr_993812", "status": "processing",
  "customer": { "email": "carlos.m@example.com", "first_name": "Carlos", "last_name": "Mendoza" },
  "items": [...], "pricing": { "subtotal": 89.00, "discount": 8.90, "shipping_cost": 0.00, "total": 80.10 },
  "payment": { "gateway": "stripe", "transaction_id": "ch_3M8x2y3z", "status": "paid" }
}
```

2. INFRAESTRUCTURA BACKEND SERVERLESS Y SEGURIDAD

El backend opera sobre arquitectura serverless con Google Cloud Platform y Firebase under el modelo Zero Trust. Toda creación de órdenes e inventario se calcula strictly en el servidor.

2.1 Reglas Globales de Seguridad de Firestore (`firestore.rules`)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function isAuthenticated() { return request.auth != null; }
    function isOwner(userId) { return isAuthenticated() && request.auth.uid == userId; }
    function isAdmin() { return isAuthenticated() && request.auth.token.admin == true; }

    match /products/{productId} {
      allow read: if true;
      allow write: if isAdmin();
      match /variants/{variantId} { allow read: if true; allow write: if isAdmin(); }
    }
    match /users/{userId} {
      allow read: if isOwner(userId) || isAdmin();
      allow create: if isOwner(userId) && !request.resource.data.keys().hasAny(['role']);
      allow update: if isOwner(userId) && !request.resource.data.diff(resource.data).affectedKeys().hasAny([
'role']);
      allow delete: if isAdmin();
    }
    match /carts/{cartId} {
      allow read, write: if isAuthenticated() && (resource == null || resource.data.user_id == request.auth.
uid || cartId == request.auth.uid);
    }
    match /orders/{orderId} {
      allow read: if isAuthenticated() && (resource.data.user_id == request.auth.uid || isAdmin());
      allow write: if false; // Bloqueado: Creación exclusiva vía Cloud Functions
    }
    match /roma_club/{emailHash} {
      allow create: if true;
      allow read, update, delete: if isAdmin();
    }
  }
}
```

2.2 Transacción Atómica de Checkout en Cloud Functions (TypeScript)

```
export const createOrderTransaction = functions.https.onCall(async (data, context) => {
  if (!context.auth) throw new functions.https.HttpsError('unauthenticated', 'Usuario no autenticado.');
  const userId = context.auth.uid;
  const { cart_id, shipping_address, coupon_code } = data;

  return await db.runTransaction(async (transaction) => {
    const cartRef = db.collection('carts').doc(cart_id);
    const cartDoc = await transaction.get(cartRef);
    if (!cartDoc.exists || cartDoc.data()?.user_id !== userId) {
      throw new functions.https.HttpsError('not-found', 'Carrito no válido.');
    }
    const items = cartDoc.data()!.items || [];
    let subtotal = 0;
    const validatedItems = [];

    for (const item of items) {
      const variantRef = db.collection('products').doc(item.product_id).collection('variants').doc(item.vari
ant_id);
      const variantDoc = await transaction.get(variantRef);
      if (!variantDoc.exists || variantDoc.data()!.stock < item.quantity) {
        throw new functions.https.HttpsError('resource-exhausted', `Stock insuficiente para ${item.title}`);
      }
      // Descuento Atómico de Inventario
      transaction.update(variantRef, { stock: variantDoc.data()!.stock - item.quantity });
      subtotal += variantDoc.data()!.price * item.quantity;
      validatedItems.push({ ...item, price: variantDoc.data()!.price });
    }

    const discount = coupon_code === 'ROMA10OFF' ? subtotal * 0.10 : 0;
    const total = subtotal - discount;

    const orderRef = db.collection('orders').doc();
    transaction.set(orderRef, {
      order_id: orderRef.id, user_id: userId, status: 'processing',
      shipping_address, items: validatedItems,
      pricing: { subtotal, discount, shipping_cost: 0, total },
      created_at: admin.firestore.FieldValue.serverTimestamp()
    });
    transaction.update(cartRef, { items: [], subtotal: 0 });
    return { success: true, order_id: orderRef.id, total };
  });
});
```

3. AUDITORÍA Y PRUEBAS UNITARIAS DE SEGURIDAD EN EMULADOR

Para garantizar que ninguna regla de seguridad contenga vacíos, se implementó la suite de tests en Jest utilizando `@firebase/rules-unit-testing` sobre el emulador local de Firestore.

```
describe(' Auditoría de Reglas de Seguridad Firestore', () => {
  test('Permite lectura pública de catálogo', async () => {
    const db = testEnv.unauthenticatedContext().firestore();
    await assertSucceeds(db.collection('products').doc('prod_8829').get());
  });

  test('Bloquea auto-elevación a rol Admin', async () => {
    const db = testEnv.authenticatedContext('usr_carlos').firestore();
    await assertFails(db.collection('users').doc('usr_carlos').update({ role: 'admin' }));
  });

  test('Bloquea creación directa de órdenes desde el cliente', async () => {
    const db = testEnv.authenticatedContext('usr_carlos').firestore();
    await assertFails(db.collection('orders').doc('ord_hack').set({ user_id: 'usr_carlos', total: 0.01 }));
  });
});
```

4. ARQUITECTURA MÓVIL EN FLUTTER (IOS & ANDROID)

La aplicación móvil se estructura en Flutter utilizando el patrón Feature-First (Clean Architecture). A continuación se presenta la jerarquía de carpetas y los archivos iniciales de configuración.

4.1 Estructura de Carpetas Local (`lib/`)

```
lib/
├── main.dart                     # Punto de entrada (Firebase + Riverpod + Theme)
├── app/
│   ├── router/ app_router.dart     # Enrutamiento declarativo con GoRouter
│   └── theme/
│       ├── app_colors.dart         # Blanco Puro (#FFFFFF), Negro Carbono (#1E232A), Dorado (#D4A843)
│       └── app_theme.dart          # Light Mode & Dark Mode (WCAG Compliance)
├── core/
│   ├── constants/ & utils/         # Validadores y formateadores de moneda
│   ├── firebase/ fcm_service.dart  # Notificaciones Push FCM y Deep Linking
│   └── widgets/                    # StickyCTA, FullWidthGallery, PulsingHotspot
└── features/
    ├── catalog/                    # PLP: Filtros y Grilla de Productos
    ├── product_detail/             # PDP: Galería y Sticky CTA
    ├── get_the_look/               # Cross-Selling Interactivo con Hotspots
    ├── cart/ & checkout/           # Carrito Persistente y Checkout de 3 Pasos
    └── orders/ & profile/          # Historial de Pedidos y Conmutador Dark Mode
```

4.2 Archivo de Configuración de Sistema Visual (`app_theme.dart`)

```
abstract class AppColors {
  static const Color blancoPuro = Color(0xFFFFFFFF);
  static const Color negroCarbono = Color(0xFF1E232A);
  static const Color doradoCalidez = Color(0xFFD4A843);
  static const Color darkSurface = Color(0xFF252B33);
}

abstract class AppTheme {
  static ThemeData get lightTheme => ThemeData(
    useMaterial3: true, brightness: Brightness.light,
    scaffoldBackgroundColor: AppColors.blancoPuro,
    colorScheme: const ColorScheme.light(primary: AppColors.doradoCalidez, onSurface: AppColors.negroCarbono
),
  );

  static ThemeData get darkTheme => ThemeData(
    useMaterial3: true, brightness: Brightness.dark,
    scaffoldBackgroundColor: AppColors.negroCarbono,
    colorScheme: const ColorScheme.dark(primary: AppColors.doradoCalidez, surface: AppColors.darkSurface),
  );
}
```

5. IMPLEMENTACIÓN DE MÓDULOS DE INTERFAZ Y COMPONENTES

5.1 Componente Sticky CTA (`sticky_cta.dart`)

```
class StickyCtaWidget extends StatelessWidget {
  final double price;
  final VoidCallback onAddToCart;
  const StickyCtaWidget({super.key, required this.price, required this.onAddToCart});

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 14),
      decoration: BoxDecoration(color: Theme.of(context).scaffoldBackgroundColor),
      child: SafeArea(
        top: false,
        child: Row(
          children: [
            Column(crossAlignment: CrossAlignment.start, children: [
              const Text('Precio Total', style: TextStyle(fontSize: 11, color: Colors.grey)),
              Text('\$$price USD', style: const TextStyle(fontSize: 20, fontWeight: FontWeight.w900)),
            ]),
            const SizedBox(width: 24),
            Expanded(
              child: SizedBox(
                height: 52,
                child: ElevatedButton(
                  onPressed: onAddToCart,
                  style: ElevatedButton.styleFrom(backgroundColor: AppColors.doradoCalidez),
                  child: const Text('AÑADIR AL CARRITO', style: TextStyle(fontWeight: FontWeight.w900)),
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

5.2 Módulo 'Get the Look' & Hotspots Pulsantes

```
class PulsingHotspotWidget extends StatefulWidget {
  final VoidCallback onTap;
  const PulsingHotspotWidget({super.key, required this.onTap});
  @override State<PulsingHotspotWidget> createState() => _PulsingHotspotWidgetState();
}

class _PulsingHotspotWidgetState extends State<PulsingHotspotWidget> with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  @override void initState() {
    super.initState();
    _controller = AnimationController(vsync: this, duration: const Duration(seconds: 2))..repeat();
  }
  @override void dispose() { _controller.dispose(); super.dispose(); }

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: widget.onTap,
      child: AnimatedBuilder(
        animation: _controller,
        builder: (context, child) => Stack(
          alignment: Alignment.center,
          children: [
            Container(
              width: 32 * _controller.value, height: 32 * _controller.value,
              decoration: BoxDecoration(shape: BoxShape.circle, color: AppColors.doradoCalidez.withOpacity(1
.0 - _controller.value)),
            ),
            Container(width: 18, height: 18, decoration: const BoxDecoration(shape: BoxShape.circle, color:
AppColors.doradoCalidez)),
          ],
        ),
      ),
    );
  }
}
```

6. SISTEMA DE NOTIFICACIONES PUSH (FCM)

El sistema de notificaciones gestiona alertas transaccionales, lanzamientos exclusivos de colecciones (Drops) y recuperación de carritos mediante Firebase Cloud Messaging y Deep Linking.

```
class FcmService {
  final FirebaseMessaging _fcm = FirebaseMessaging.instance;

  Future<void> initialize() async {
    await _fcm.requestPermission();
    await _fcm.subscribeToTopic('drops_global');

    FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage message) {
      final String? deepLink = message.data['deep_link'];
      if (deepLink != null) {
        // Enrutamiento directo al producto o pedido
        appRouter.push(deepLink);
      }
    });
  }
}
```

7. PIPELINE DE INTEGRACIÓN CONTINUA (GITHUB ACTIONS)

El flujo automatizado `.github/workflows/flutter_ci_cd.yml` ejecuta el análisis estático, las pruebas unitarias y la compilación paralela de ejecutables Android e iOS.

```
name:  RomaSupply Mobile CI/CD Pipeline
on:
  push: { branches: [ main, develop ] }
  pull_request: { branches: [ main ] }

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with: { flutter-version: '3.22.x', channel: 'stable', cache: true }
      - run: flutter pub get
      - run: flutter analyze
      - run: flutter test --coverage

  build_android:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
      - run: flutter build apk --release --split-per-abi
      - run: flutter build appbundle --release
      - uses: actions/upload-artifact@v4
        with: { name: romasupply-android-builds, path: 'build/app/outputs/**/*.aab' }
```

8. HOJA DE RUTA DE EJECUCIÓN MASTER (10 SEMANAS)

Plan secuencial de ejecución estructurado en 7 fases para evitar dependencias circulares y asegurar la calidad del despliegue en producción.

Fase	Semanas	Objetivos Principales	Entregables Clave
FASE 1	Semanas 1 - 2	Aprovisionamiento GCP/Firebase y Emulador Local	Proyectos dev/prod, Auth, Firestore y Storage
FASE 2	Semanas 2 - 3	Modelo NoSQL, Reglas Zero Trust y Cloud Functions	firestore.rules, tests Jest y createOrderTransaction
FASE 3	Semanas 3 - 4	Base Flutter, Clean Architecture y Design System	AppColors, AppTheme, GoRouter y Riverpod
FASE 4	Semanas 5 - 7	Módulos Core UI/UX (PLP, PDP, Get the Look, Checkout)	Sticky CTA, Hotspots, Checkout 3 Pasos y Perfil
FASE 5	Semana 8	Notificaciones Push FCM y Deep Linking	fcm_service.dart, canal Drops Global y Deep Links
FASE 6	Semana 9	Pipeline CI/CD en GitHub Actions & QA	Workflow .yml, tests automáticos y Firebase App Check
FASE 7	Semana 10	Publicación en Google Play, App Store y Launch	Archivos .aab/.ipa en producción y Crashlytics

FIN DEL MANUAL TÉCNICO Y GUÍA DE EJECUCIÓN

Este manual técnico constituye el documento definitivo de especificación y guía de ejecución para el equipo de desarrollo. Todos los módulos han sido verificados contra los requisitos de diseño e infraestructura de RomaSupply.